Chapitre 2 : Variables, Constantes et Types de Données en Java

---

Par Robert DIASSÉ

Objectifs du chapitre

- Comprendre le rôle des variables dans un programme.
- Différencier variables et constantes.
- Manipuler les principaux types de données en Java.
- Déclarer, initialiser et utiliser des variables.
- Respecter les conventions de nommage en Java.

2.1 Qu'est-ce qu'une variable ?

Une **variable** est un espace mémoire réservé pour stocker une valeur.

- Chaque variable a un **nom** (identifiant), un **type** (nature de la donnée qu'elle peut contenir) et une **valeur**.
- On peut modifier la valeur d'une variable au cours de l'exécution du programme.

Exemple simple :

```java
public class ExempleVariable {
    public static void main(String[] args) {
        int age = 20; // Déclaration + initialisation d'une variable entière
        System.out.println("J'ai " + age + " ans");
    }
}
```

Explication :

- `int age = 20;` → création d'une variable entière appelée `age` qui contient la valeur 20.
- `System.out.println("J'ai " + age + " ans");` → concatène la chaîne de caractères avec la valeur de la variable `age` et l'affiche.

Sortie :

```
J'ai 20 ans
```

2.2 Déclaration et initialisation

En Java, la syntaxe pour déclarer une variable est la suivante :

```
type nomDeVariable = valeurInitiale;
```

- **type** → précise le type de donnée (int, double, String, etc.).
- **nomDeVariable** → identifiant choisi par le programmeur.
- **valeurInitiale** → valeur de départ.

On peut aussi séparer la **déclaration** et l'**initialisation** :

```
int nombre;         // Déclaration
nombre = 15;        // Initialisation
```

2.3 Les types de données en Java

Java distingue deux grandes familles de types :

1. **Types primitifs** (simples et très utilisés).
2. **Types objets (ou références)** (issus des classes).

2.3.1 Les types primitifs

Voici les principaux types de données primitifs :

Type	Taille (bits)	Exemple	Description
`byte`	8	100	Petit entier (-128 à 127)
`short`	16	20000	Entier court
`int`	32	150000	Entier standard
`long`	64	10000000000L	Très grand entier
`float`	32	3.14f	Nombre réel simple précision
`double`	64	3.1415926535	Nombre réel double précision
`char`	16	'A'	Caractère Unicode

Type	Taille (bits)	Exemple	Description
`boolean`	1	true / false	Valeur logique

Exemple d'utilisation :

```
public class ExempleTypes {
    public static void main(String[] args) {
        int entier = 42;
        double reel = 3.14;
        char lettre = 'J';
        boolean estVrai = true;

        System.out.println("Entier : " + entier);
        System.out.println("Réel : " + reel);
        System.out.println("Caractère : " + lettre);
        System.out.println("Booléen : " + estVrai);
    }
}
```

Explication :

- `int entier = 42;` → stockage d'un nombre entier.
- `double reel = 3.14;` → stockage d'un nombre décimal.
- `char lettre = 'J';` → stockage d'un caractère.
- `boolean estVrai = true;` → stockage d'une valeur logique.

Sortie :

```
Entier : 42
Réel : 3.14
Caractère : J
Booléen : true
```

2.3.2 Les types objets (références)

En plus des types primitifs, Java possède des **types objets**, issus de classes.
Exemples fréquents :

- `String` (chaînes de caractères).
- `Integer`, `Double` (classes enveloppes pour les primitifs).

Exemple :

```
public class ExempleString {
    public static void main(String[] args) {
        String nom = "Alice";
```

```
        System.out.println("Bonjour " + nom);
    }
}
```

Explication :

- `String nom = "Alice";` → création d'une variable objet de type `String`.
- Elle contient une chaîne de caractères (texte).

Sortie :

```
Bonjour Alice
```

2.4 Les constantes

Une **constante** est une variable dont la valeur ne peut pas changer pendant l'exécution du programme.
On utilise le mot-clé `final` pour déclarer une constante.

Exemple :

```java
public class ExempleConstante {
    public static void main(String[] args) {
        final double PI = 3.14159;
        System.out.println("La valeur de PI est : " + PI);
    }
}
```

Explication :

- `final double PI = 3.14159;` → déclaration d'une constante `PI`.
- Une fois initialisée, sa valeur ne peut pas être modifiée.

Sortie :

```
La valeur de PI est : 3.14159
```

2.5 Les opérateurs en Java

En Java, les **opérateurs** sont des symboles spéciaux qui permettent d'effectuer des opérations sur des variables et des valeurs. Ils transforment les données, comparent des expressions ou contrôlent le flux logique d'un programme.
On distingue plusieurs catégories d'opérateurs.

---

1. Les opérateurs arithmétiques

Ils servent à effectuer des calculs mathématiques.

Opérateur	Nom	Exemple	Résultat
`+`	Addition	`5 + 3`	`8`
`-`	Soustraction	`10 - 4`	`6`
`*`	Multiplication	`7 * 2`	`14`
`/`	Division entière	`10 / 3`	`3` (car ce sont des entiers)
`%`	Modulo (reste de division)	`10 % 3`	`1`

⚠ **Attention aux erreurs fréquentes** :

- Si les deux opérandes sont des entiers (`int`), la division donne toujours un entier (les décimales sont perdues).
  Exemple : `7 / 2 = 3` et non `3.5`.
- Pour obtenir un résultat réel, il faut utiliser `double` :

```
double resultat = 7.0 / 2; // résultat = 3.5
```

---

2. Les opérateurs d'incrémentation et de décrémentation

Ils permettent d'augmenter ou de diminuer une valeur de **1**.

Opérateur	Exemple	Explication
`++`	`i++` ou `++i`	Incrémente la variable `i` de 1
`--`	`i--` ou `--i`	Décrémente la variable `i` de 1

Différence entre `i++` et `++i` :

- `i++` (post-incrémentation) : renvoie d'abord la valeur, puis incrémente.
- `++i` (pré-incrémentation) : incrémente d'abord, puis renvoie la valeur.

Exemple :

```
int i = 5;
System.out.println(i++); // affiche 5, puis i devient 6
System.out.println(++i); // incrémente à 7, puis affiche 7
```

---

3. Les opérateurs relationnels

Ils comparent deux valeurs et renvoient un **booléen** (`true` ou `false`).

Opérateur	Signification	Exemple	Résultat
`==`	Égal à	`5 == 5`	`true`
`!=`	Différent de	`5 != 3`	`true`
`>`	Supérieur à	`7 > 4`	`true`
`<`	Inférieur à	`2 < 8`	`true`
`>=`	Supérieur ou égal	`6 >= 6`	`true`
`<=`	Inférieur ou égal	`3 <= 5`	`true`

⚠ **Erreur fréquente à éviter** :
Ne pas confondre `==` (comparaison) et `=` (affectation).
Exemple incorrect :

```
if (a = 5) // Erreur
```

Il faut écrire :

```
if (a == 5) // Comparaison
```

---

4. Les opérateurs logiques

Ils combinent plusieurs conditions logiques.

Opérateur	Signification	Exemple	Résultat
`&&`	ET logique	`(5 > 3 && 8 > 6)`	`true`
`||`	OU logique	`(5 > 3 || 8 < 6)`	`true`
`!`	NON logique	`!(5 > 3)`	`false`

---

5. Les opérateurs d'affectation

Ils servent à affecter une valeur à une variable. Certains combinent affectation et opération.

Opérateur	Exemple	Équivalent
`=`	`a = 5`	affecte 5 à `a`
`+=`	`a += 3`	`a = a + 3`
`-=`	`a -= 2`	`a = a - 2`

Opérateur	Exemple	Équivalent
`*=`	`a *= 4`	`a = a * 4`
`/=`	`a /= 2`	`a = a / 2`
`%=`	`a %= 2`	`a = a % 2`

---

6. Les opérateurs sur les bits (avancé)

Ces opérateurs s'appliquent directement sur les **bits** des entiers.
Ils sont moins utilisés en algorithmique de base, mais très puissants.

Opérateur	Description	Exemple
`&`	ET binaire	`6 & 3` → `2`
`|`	OU binaire	`6 | 3`→`7`
`^`	OU exclusif (XOR)	`6 ^ 3` → `5`
`~`	NON binaire (complément)	`~6` → `-7`
`<<`	Décalage à gauche	`6 << 1` → `12`
`>>`	Décalage à droite	`6 >> 1` → `3`

2.6 Conventions de nommage

- Les noms de variables commencent par une lettre (pas de chiffre au début).
- Utiliser la **camelCase** : `monNombre`, `ageEtudiant`.
- Les constantes sont en **MAJUSCULES** avec des underscores : `NOMBRE_MAX`.